

CSE 598: Combinatorial Optimization for Graph Algorithms

Zhengjie Min

September 1, 2026

Abstract

This is a template for TCS (and other kind of) lecture notes.

Contents

I	Flow Algorithms	2
1	Introduction	3
A	Algebra	5
B	Analysis	6

Part I

Flow Algorithms

Chapter 1

Introduction

In this part we'll investigate 4 min-max theorems. In these theorems, a max property of a graph is often proved to be equal to a min property of the graph.

We should be aware that such min-max theorems are not seen everywhere. For example:

Claim.

$$\text{Max Matching} \leq \text{Min Vertex Cover}$$

Proof. Let M be a matching and C be a vertex cover. Then each edge in M must have at least one endpoint in C . Since the edges in M are disjoint, we have that $|M| \leq |C|$. ■

The easiest counterexample to show that the above inequality can be strict is a triangle graph. The maximum matching has size 1, while the minimum vertex cover has size 2.

Here we first list the 4 min-max theorems and prove the $\leq$ direction. Then we will prove the tightness in the next few chapters.

Claim (Menger's Theorem). In a unweighted graph G , the maximum number of edge-disjoint s - t paths $\leq$ the minimum s - t cut (i.e. the minimum number of edges whose removal disconnects s and t).

Proof. Let $P_1, P_2, \dots, P_k$ be edge-disjoint s - t paths. Let C be an s - t cut. Then each path P_i must contain at least one edge in C . Since the paths are edge-disjoint, we have that $k \leq |C|$. ■

Claim (Nash-William Theorem). The maximum number of edge-disjoint spanning trees $\leq \min_P \lfloor \frac{|E(P)|}{|P|-1} \rfloor$ over all partition P of the graph.

Proof. Let $T_1, T_2, \dots, T_k$ be edge-disjoint spanning trees. Let P be a partition of the graph. Then each spanning tree T_i must contain at least $|P| - 1$ edges in $E(P)$ as a spanning tree must touch all the components of the partition P . Since the spanning trees are edge-disjoint, we have that $k(|P| - 1) \leq |E(P)|$, which implies that $k \leq \lfloor \frac{|E(P)|}{|P|-1} \rfloor$. ■

Note. Intuitively we want to find an equality between the maximum number of disjoint spanning trees and the minimum global cut. However one can find the counterexample of it in C_4 , with the maximum number of disjoint spanning trees being 1, while the minimum global cut is 2.

Definition 1.0.1. A s -arborescence in a directed graph is a spanning tree rooted at s such that all vertices are reachable from s .

Claim (Rooted Connectivity Theorem). The maximum number of edge-disjoint s -arborescences $\leq$ the minimum s cut (i.e. the minimum number of edges whose removal disconnects s and any vertex).

We won't be focusing on this theorem. Instead we want to prove a weaker version in the next few chapters:

Appendix

Appendix A

Algebra

Theorem A.0.1. For any polynomial $A \in \mathbb{C}[X]$ of degree $n - 1$, any point set $\{p_0, \dots, p_{n-1}\}$ of size n , $\{(p_i, A(p_i))\}_{i=0}^{n-1}$ determines $A(x)$ and vice versa.

Theorem A.0.2 (Fundamental Theorem of Algebra). Every non-constant single-variable polynomial with complex coefficients has at least one complex root.

In math terms, for $A(x) = \sum_{i=0}^{n-1} a_i x^i$, either $\exists x \in \mathbb{C}$ such that $A(x) = 0$, or $a_0 = a_1 = \dots = a_{n-1} = 0$.

Appendix B

Analysis

Theorem B.0.1 (Master Theorem (simplified)). Let the recurrence relation be:

$$T(n) = aT\left(\frac{n}{b}\right) + n^d$$

where $a \geq 1$ is the number of subproblems, $b > 1$ is the factor by which the problem size is reduced, and $d \geq 0$. The asymptotic bound of $T(n)$ is given by:

1. If $\log_b a > d$, then $T(n) = O(n^{\log_b a})$.
2. If $\log_b a = d$, then $T(n) = O(n^d \log n)$.
3. If $\log_b a < d$, then $T(n) = O(n^d)$.

Theorem B.0.2 (Master Theorem). Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence:

$$T(n) = aT(n/b) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

- **Case 1:** If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- **Case 3:** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (Regularity Condition), then $T(n) = \Theta(f(n))$.